Backends for Frontends (BFF) Pattern

Category: Edge / routing **One-liner:** Instead of one shared backend serving every kind of client, give each client type its own backend tailored to exactly what that client needs.

Origin: Coined by **Sam Newman**, to solve building backends for structurally different clients — mobile apps, web apps, and other client-side **frameworks** (foundations you build an app on, such as React or SwiftUI).

Part 0 — Concepts you need first

What is a "frontend" and a "backend"?

The **frontend** is what the user sees — mobile app, website, smart-TV interface. The **backend** is the server-side software it calls. A **client** here means a frontend application; "client types" are categories: mobile, web, TV, watch.

What is over-fetching?

Over-fetching is a client downloading more data than it needs. A mobile product list shows thumbnail, name and price, but the API returns the full product — a 2,000-word description, fifteen high-res image **URLs**, full specs, seller history. Why it hurts:

- **Bandwidth** — on a **metered** plan (charged per MB), it's the user's money
- **Latency** — larger **payloads** (the data in a request/response) take longer on slow links
- **Battery and CPU** — the phone must **parse JSON** (the common text data format) it then discards
- **Memory** — constrained devices hold data they never use

What is an API?

The contract for how software asks other software for something (defined in Strangler Fig). "One uniform set of APIs for all frontends" means every client gets the same **endpoints** (URLs a server answers, like `GET /products/42`) returning the same shapes.

What is aggregation?

One component calls several sources and returns a single merged result — product details, reviews and stock from three services in one response, so the client makes one request instead of three.

Why round trips matter so much on mobile

A **round trip** is one request out and its response back: **5–20 ms** on an office network, **100–300 ms or worse** on mobile. Six round trips on desktop are barely noticeable; on mobile they can be two seconds of nothing. This one fact drives most of the chapter.

Part 1 — The core idea, in plain terms

A company has three platforms: **a detailed website** (extensive information), **a mobile app** (quick essentials on the go), **a smartwatch app** (core functions only). The website can display a thousand words; the watch about eight.

Traditionally one backend serves all three, and the backend complexity appropriate for the website **overwhelms** the simpler needs of mobile and watch. The website wins by default — usually the biggest, oldest client — and everyone else gets its leftovers. BFF gives each its own backend (the course's image: each section of a diverse restaurant gets its own kitchen).

Netflix does exactly this — browsers, mobile apps and smart TVs each get their own backend service, because each device has different capabilities and interfaces.

Part 2 — The problem in detail

One-size-fits-all

A traditional backend is a single entity serving all frontends, exposing **one uniform set of APIs regardless of each client's requirements**:

	Desktop	Mobile
Processing power	High	Limited
Display	Large	Small
Network	Usually fast, unmetered	Possibly slow, possibly metered

Should mobile receive the same data as desktop, and process it all on-device? **Not quite.**

Performance inefficiency and over-fetching

Over-fetching causes slow loads and bandwidth waste. A product-list screen: **shared API response 250 KB**; **mobile displays ~4 KB** (name, price, thumbnail). The user paid for 250 KB and used 4 — seconds of waiting on a slow connection for data that's thrown away.

Complexity in code management

One codebase serving many frontends becomes **bloated** — harder to maintain, troubleshoot and update — and endpoints fill with **conditionals**:

```
if (clientType.equals("mobile")) {
    // trim the description
} else if (clientType.equals("tv")) {
    // use high-res images
} else if (clientType.equals("watch")) {
    // just the title
}
```

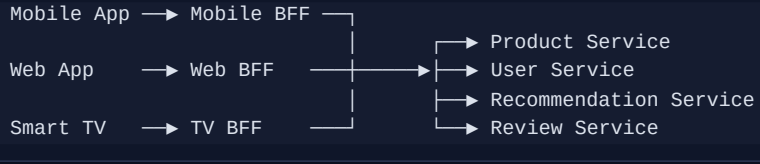
Every client's needs tangled into one method nobody can change safely.

Lack of flexibility and scalability

Two symptoms that signal you need BFF:

1. **Sifting through a monolithic codebase to change one small feature for one frontend**
2. **Scaling the entire backend because one frontend has high traffic** — a TV advert spikes mobile traffic, and you pay to scale capacity serving web and internal tools that aren't busy, because the parts can't scale separately.

Part 3 — Architecture



Three layers: **(1) Frontend applications** (desktop, mobile, web) → **(2) BFFs**, each paired with one frontend → **(3) Microservices** the BFFs call for data.

The important structural point: **the microservices are shared; only the BFF layer is duplicated.** There is still exactly one Product Service holding product truth — the **source of truth** (the authoritative copy). BFFs don't own data; they reshape it.

What a BFF actually does

- **Tailored data delivery** — acts as a **filter** so its frontend gets *just* what it needs.
- **Streamlining communication** — calls several microservices, consolidates, and presents one shape (an "interpreter" between many services and one client).
- **Handling client-specific logic** — absorbs computation the client would otherwise do.

> *Why server-side logic matters especially on mobile:* you can't force app updates. Display logic in the app means a release, app-store review, and waiting for users to update — some never will. In the BFF, you fix and **deploy** it in minutes.

Part 4 — The benefits, explained

The course lists seventeen, grouped here:

Theme	Benefit	Why
Fit & UX	Diverse frontend needs	Acknowledges that mobile, web and TV genuinely differ
	Enhanced user experience	Each device served to its capabilities
	Localization / internationalization	A BFF can serve region-specific language and features
Performance	Performance optimization	Tune data and interactions per device (slow networks, weak CPUs)
	Reduced bandwidth	Sends only what that frontend needs
	Independent scalability	A mobile spike scales only the Mobile BFF
Frontend simplicity	Simplified frontend logic	One tailored backend instead of many services
	Consistency	A **stable API while underlying microservices change** — backend teams can reorganise without breaking apps
	Versioning control	Each frontend version gets appropriate logic — acute on mobile, where old versions persist
Team & delivery	Agility	Teams ship their BFF independently; mobile doesn't queue behind web
	Easier maintenance/upgrades	Change the mobile backend without touching web
	Cross-functional teams	One team owns **a frontend and its backend end to end**
Capability	Security	Per-client auth mechanisms (e.g. cookies vs tokens)
	Error handling & resilience	Per-frontend error handling — web may degrade gracefully where mobile can't
	A/B testing	Test on one client type **without affecting others**
	Feature flagging	Toggle features per frontend at the backend
	Handling legacy systems	A **bridge** from new frontends to legacy during migration (*Strangler Fig*)

Part 5 — Worked example: e-commerce product details

One shared **Product Service**, a **Web BFF** (extensive detail), a **Mobile BFF** (concise, to save data and load faster).

1. Product Service — the real backend

```
@RestController
@RequestMapping("/products")
public class ProductServiceController {

    @Autowired
    private ProductRepository productRepository;

    @GetMapping("/{productId}")
    public Product getProductById(@PathVariable String productId) {
        return productRepository.findById(productId)
                .orElseThrow(() -> new ProductNotFoundException(productId));
    }
}
```

Reading it (Spring; `@` words are **annotations** — labels telling the framework how to treat code):

- `@RestController` — a web handler returning data, not **HTML** pages
- `@Autowired` — Spring **injects** the `ProductRepository` (the class that reads/writes this data in the database)
- `@RequestMapping("/products")` + `@GetMapping("/{productId}")` — handles `GET /products/123`; `@PathVariable` pulls `123` into the parameter
- `orElseThrow` — no product → a clear error instead of returning `null`

This service owns the **full** `Product` and knows nothing about mobile or web.

2. Web BFF — the detailed view

```
@RestController
@RequestMapping("/web/products")
public class WebProductController {

    @Autowired
    private ProductServiceClient productServiceClient;

    @GetMapping("/{productId}")
    public ProductDetails getProductDetails(@PathVariable String productId) {
        Product product = productServiceClient.getProductById(productId);
        return convertToWebProductDetails(product);
    }

    private ProductDetails convertToWebProductDetails(Product product) {
        // Convert to a detailed view suitable for web:
        // richer descriptions, full-size images, etc.
    }
}
```

The prefix `/web/products` gives each BFF its own namespace, so clients can't call the wrong one. `ProductServiceClient` is a **Feign client** (you declare an interface; the library generates the HTTP-calling code, so `getProductById(id)` looks like a normal method call) or any HTTP client.

3. Mobile BFF — the concise view

Identical, except `@RequestMapping("/mobile/products")`, return type `SimpleProductDetails`, and a `convertToMobileProductDetails` that strips descriptions and shrinks images.

The structural point — read this part twice

The two BFFs are **nearly identical**: both call the **same** `productServiceClient.getProductById(productId)` and receive the **same** full `Product`. They differ **only in the transformation step** and therefore the return type (Web: `ProductDetails`, rich; Mobile: `SimpleProductDetails`, stripped).

A BFF is a projection layer, not a second source of truth. (A **projection** is a reshaped view of data that lives elsewhere.) One Product Service holds one truth; each BFF projects a different view of it. A BFF *storing* data or deciding what a product *is* has stopped being a BFF.

Part 6 — Problems with BFF, and how to handle them

Problem	Solution
1. Code duplication across BFFs — auth handling, error formats, logging	**Shared libraries**, or **internal microservices** the BFFs consume
2. Increased maintenance — three BFFs = three **deployment pipelines**, three sets of **dependencies** to patch	**DevOps** practices (dev + ops working together, heavy automation): automated testing, deployment, monitoring
3. Coordination overhead between frontends and their BFFs in large teams	**Clear contracts** and **consumer-driven contract testing**

Consumer-driven contract testing: the consumer (the app) writes its expectations of the BFF as an executable test that runs in the BFF's build. If the BFF breaks something the app relies on, the BFF's build fails — before production, not after.

Performance implications

Implication	Solution
Network latency — an extra hop	Optimise network paths; efficient **protocols** (e.g. HTTP/2, gRPC); place **BFFs geographically close to their frontends**
Resource utilization — each BFF needs its own resources	Optimise; **serverless** (provider runs and scales servers, pay per call) where apt; scale on demand
Caching challenges — same data cached in three shapes	Caching at both BFF and frontend; good use of `Cache-Control` **headers** (how long a response may be reused)
Load on backend services — three BFFs fetching the same product = three calls	Scalable, monitored backends; **rate limiting** if needed

> ⚠️ **The duplication trap is the one that actually bites.** Three BFFs slowly drifting apart on auth or error semantics is how "tailored" becomes "inconsistent" — users hit a bug on mobile that doesn't exist on web. Shared libraries are not optional at scale.

Part 7 — System design example: Netflix

Netflix serves phones, tablets, smart TVs, laptops and consoles. **Scenario: viewing movie details** — title, description, genre, rating, cast, recommendations, reviews.

BFF	Device priority	Responsibilities
Mobile BFF (phones, tablets)	Quick loading, less data	**Authentication** (logged in, valid subscription) · **routing** to Movie Details and Recommendation services · **aggregation** · **transformation**: smaller images, shorter descriptions, **a subset of recommendations**
TV BFF	Rich visuals, detail, high-quality images	**Authentication & authorization** — not just login but **whether the device is authorized to play HD content** · routing & aggregation tuned to TV capabilities and networks · high-res images, full cast, elaborate descriptions, full recommendations
Web BFF (browsers)	Balance of performance and richness	Standard login check · routing & aggregation for browsers · content for a wide range of browsers and screen sizes

Benefits realised: tailored UX per device, streamlined data and faster loads, **maintainability** — frontend teams iterating fast with their own BFF. **Challenges realised:** maintaining every BFF, and keeping behaviour consistent across them.

> **The TV BFF's HD-entitlement check is the clearest single argument for BFF over one shared gateway.** The rule is meaningless for phones and browsers. In a shared gateway it would sit behind a device-type conditional, polluting a component every team touches; in the TV BFF it's simply that BFF's business.

Part 8 — BFF vs. API Gateway

	API Gateway	BFF
How many	One shared entry point	One per client type
Optimised for	All clients generically	One specific client
Client-specific logic	Pollutes it	Belongs there
Main risk	Bottleneck / distributed monolith	Duplication and drift
Owned by	Usually a platform team (builds shared infrastructure)	Usually the frontend team

They are not mutually exclusive. A very common production setup is a thin shared gateway doing **TLS termination** (decrypting HTTPS at the edge), coarse authentication and rate limiting, with BFFs behind it: the gateway does what's identical for everyone, the BFFs do what differs.

Think of BFF as API Gateway specialised per client — and note the ownership difference: being owned by the frontend team is a large part of why BFF delivers agility.

Part 9 — Common beginner misconceptions

- **"Each BFF needs its own database."** No. BFFs are generally **stateless** (keep no data between requests); one with a database has usually become a microservice by accident.
- **"BFF means one backend per screen."** No — per **client type**. All mobile screens share the Mobile BFF.
- **"You need BFF as soon as you have a mobile app."** Only when needs genuinely diverge. If mobile and web want nearly the same data, a shared gateway is simpler; adopt BFF when you start adding device-conditionals to shared code.
- **"GraphQL makes BFF obsolete." GraphQL** is an API query language where one endpoint accepts queries naming exactly the fields wanted — a genuine alternative for *shaping* (it solves over-fetching). It doesn't solve client-specific *logic* (the HD check), independent scaling, or ownership. Some teams use one GraphQL layer per client type.

- **"BFFs make the system faster."** Not inherently — they add a hop. They make it faster *for the client* by cutting round trips and payload size, which dominates on mobile.

Part 10 — Check your understanding

1. What is over-fetching, and why does it matter more on mobile than desktop?
2. In the worked example, what is the **only** thing that differs between Web BFF and Mobile BFF?
3. Why is it wrong for a BFF to own a database?
4. What's the strongest argument for the TV BFF over a shared gateway?
5. Name the main cost of BFF and its standard mitigation.
6. When should you *not* use BFF?

(Answers: 1 — downloading more data than needed; mobile has metered data, slow networks, limited CPU/memory; 2 — the transformation step and its return type, both call the same service for the same object; 3 — BFFs are projection layers, not sources of truth — one service owns each piece of data; 4 — the HD-entitlement authorization check is meaningless to other clients and would pollute shared code; 5 — code duplication across BFFs, mitigated by shared libraries and internal services; 6 — when clients' needs are nearly identical, or the team is too small to maintain several backends.)

Part 11 — Interview soundbite (30 seconds)

"BFF, from Sam Newman, gives each client type its own backend instead of sharing one. It solves over-fetching and one-size-fits-all APIs — a mobile app on a metered connection shouldn't receive a desktop payload or need six round trips on a slow network for one screen. Each BFF calls the same microservices but aggregates and transforms for its client: small images and short descriptions for mobile, high-res and full cast for TV. Crucially the BFF is a projection layer, not a source of truth — the microservices still own the data. Netflix does this across phones, TVs and browsers, and their TV BFF adds an HD-entitlement check — client-specific logic that would pollute a shared gateway. It also lets frontend teams own and ship their backend independently and scale per client. Costs: code duplication, more codebases, an extra hop, and keeping behaviour consistent — so shared libraries and consumer-driven contract testing matter. It's essentially API Gateway specialised per client, and the two are often combined."

Part 12 — Implementation

A. Runnable demo

Full source: `demos/BffDemo.java`

```
cd demos && ./gradlew runCh04
```

Three BFFs, one **upstream** service (the one a component calls for data), three projections:

```
// Mobile: trim the copy, shrink the image
new MobileProductDetails(
    p.id(), p.name(), p.price(),
    truncate(p.longDescription(), 120),
    p.hiResImages().get(0).replace("4k", "thumb"));

// Watch: two fields, nothing else
new WatchProductDetails(truncate(p.name(), 20), p.price());
```

B. What you should see

```
=== 1. The over-fetching problem ===
  shared API payload:   3,525 bytes
  >> a watch showing name + price downloads ALL of that

=== 2. Each BFF projects the same object differently ===
  Web BFF:      3,525 bytes  (100% of shared)
  Mobile BFF:     251 bytes  (  7% of shared)
  Watch BFF:       31 bytes  (  0% of shared)
  >> mobile saves ~3274 bytes PER PRODUCT

=== 3. All BFFs call the SAME upstream service ===
  ProductService invocations so far: 4
  >> BFFs are projection layers - one source of truth remains

=== 4. Client-specific logic belongs in its own BFF ===
  certified TV:     Full HD payload ... with 3 4K images
  uncertified TV:   SD-only payload ...
```

Mobile receives 7% of the shared payload — across a 50-item list, the difference between instant and visibly stalling on a train. Section 3 is the conceptual check: every BFF called the *same* `ProductService`; **they add and own no data**.

C. Production — a BFF is just a small service

No special framework — an ordinary application whose job is aggregation and shaping. `CompletableFuture.supplyAsync` runs each call on a background thread (parallel), `.exceptionally(...)` supplies a default on failure, `join()` waits:

```java
@RestController
@RequestMapping("/mobile/v1")
public class MobileBffController {

    private final ProductClient products;
    private final ReviewClient reviews;
    private final InventoryClient inventory;

    @GetMapping("/products/{id}")
    public MobileProductView get(@PathVariable String id) {
        // Fan out in parallel - the BFF exists to save the client round trips
        var pf = CompletableFuture.supplyAsync(() -> products.byId(id));
        var rf = CompletableFuture.supplyAsync(() -> reviews.summaryFor(id))
                    .exceptionally(e -> ReviewSummary.empty());     // degrade, don't fail
        var inf = CompletableFuture.supplyAsync(() -> inventory.stockFor(id))
                    .exceptionally(e -> Stock.unknown());

        CompletableFuture.allOf(pf, rf, inf).join();
        return MobileProductView.from(pf.join(), rf.join(), inf.join());
    }
}
```

GraphQL as an alternative for the *shaping* half — clients request exactly their fields, so one endpoint serves all without over-fetching:

```
query MobileProduct($id: ID!) {
  product(id: $id) { id name price thumbnail }     # mobile asks for four fields
}
```

It does **not** solve client-specific logic, independent scaling, or ownership; a common setup is **one GraphQL BFF per client type** — shaping from GraphQL, isolation from BFF.

Shared code across BFFs — the duplication mitigation (**tracing** = recording a request's path across services):

```
bff-common/          <- auth, error format, tracing, resilience config
  ├─ mobile-bff/
  ├─ web-bff/
  └─ tv-bff/
```

D. Implementation pitfalls

1. Giving a BFF its own database. The most common mistake. BFFs are **stateless projection layers**; **persistent state** means you've created a microservice with duplicated data ownership.

2. Sequential upstream calls. Three sequential server-side calls just move the latency instead of removing it. Fan out with `CompletableFuture.allOf`.

3. No fallback on secondary data. If reviews are down the page should still render. Note `.exceptionally(...)` on reviews and inventory but *not* the product — the primary resource legitimately fails the request; supporting data must not.

4. Copy-pasting auth into every BFF. How three BFFs drift into three subtly different security behaviours. Extract to `bff-common` from the start.

5. One BFF per screen. Per **client type**. Twenty BFFs is twenty pipelines for no benefit.

6. Business rules migrating into the BFF. "Is this order refundable?" belongs in the order service so every client gets the same answer. Only **presentation** decisions belong in a BFF.

7. Versioning by breaking changes. (A **breaking change** — e.g. removing or renaming a field — makes existing clients fail.) Old mobile versions live for years: version the path (`/mobile/v1`, `/mobile/v2`) and keep old versions running — a major reason BFF exists.

Related patterns

- **API Gateway** — the shared-entry-point alternative, and frequent companion
- **Strangler Fig** — a BFF can bridge new frontends to legacy systems mid-migration
- **Service Discovery** — how BFFs locate the services they aggregate
- **Circuit Breaker** — a BFF calling several services needs protection when one fails